



Assignment Title:

Dynamic Robotics Simulation and Autonomous System Design

Programme title:

MSc Artificial Intelligence

Prepared by:

Amaan Mohammed Khalander

Module:

Robotics Software and Programming

Year:

Term 2 – 2026

Submission Date:

23 / September / 2026

Issued by:

Dr Fahmid Al Farid

Learner declaration

Item	Response
Word count	3,286
Use of proof-reader/proof-reading service	NO
If yes, specify which service	N/A
I confirm that I submit this work as my own work and have cited all sources used	YES
I confirm that I have followed guidance on the acceptable use of AI tools for this assignment	YES
Where use of appropriately cited AI tools is permitted, I confirm that I have cited in accordance with the UCA Harvard Referencing Standard	YES

Student signature	Date
	23 / September / 2026

Source Code and Relevant Project Links:

Source Code:

- Task 1:
<https://github.com/Amaan9136>
- Task 2:
<https://github.com/Amaan9136>

Table of Contents

Abstract	6
Introduction.....	7
Task 1: Interactive PyBullet Environment with Object Semantics.....	8
3.1 Project Definition.....	8
3.2 Literature Review.....	8
3.3 Design and Implementation	9
3.4 Code Implementation.....	10
3.5 Execution and Testing.....	11
3.6 Evaluation and Reflection.....	13
Task 2: Goal-Oriented Autonomous Robot with Monitoring (ROS2).....	14
4.1 Project Definition.....	14
4.2 Design and Implementation	15
4.3 Execution and Testing.....	17
4.4 Evaluation and Reflection.....	22
Combined Evaluation and Reflection	23
Concluding Remarks.....	24
Acknowledgement	25
References.....	26

Table of Figures

Figure 1: Task 1 pipeline, from the floor-plan sketch and URDF generation through the semantic layer and camera rendering to the query helpers and outputs.	9
Figure 2: Six PyBullet camera viewpoints, robot repositioned between renders while all other objects remain fixed.	12
Figure 3: ROS2 node and topic architecture of the semantic_nav_monitor package, showing operator input, the four nodes and the Gazebo simulation.	15
Figure 4: Six-state finite state machine in mission_controller, with the REVERSE, ORIENT and COMMIT sub-phases shown beneath their parent states.	16
Figure 5: TheConstruct.ai Rosject XctZpczWOA: file explorer, mission_controller.py in the code editor, the live Simulation panel, and monitor_node's summary stream in the shell.	18
Figure 6: Gazebo client mid-patrol on TheConstruct.ai: robot among the eight pallet obstacles and checkpoint markers, with elapsed real time and iteration count visible.	18
Figure 7: keyboard_hri_node in a Rosject shell: start and pause keypresses published and confirmed, alongside the live Simulation panel.	19
Figure 8: Live /mission_state echo showing the robot cycle through AVOID_OBSTACLE, REPLAN, IDLE, PLANNING, NAVIGATE and MISSION_COMPLETE.	20
Figure 9: monitor_node's mission summary, captured shortly before MISSION_COMPLETE.	20
Figure 10: Local Gazebo Harmonic verification: LIDAR rays visualised around the robot near a pallet obstacle, with the forward camera feed shown alongside.	21

List of Tables

Table 1: Semantic object registry as printed by print_object_roles() (14 objects, excluding the robot).	10
Table 2: ROS2 node architecture.	15
Table 3: Finite state machine (six states).	16
Table 4: Mission summary logged by monitor_node on TheConstruct.ai, Rosject XctZpczWOA, shortly before MISSION_COMPLETE.	20

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work, and that none of its parts has been copied from printed or electronic sources, translated from foreign sources, or reproduced from essays by other researchers or students. Wherever I have based my work on ideas or other people's texts, I clearly declare it through proper references in line with academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname:

AMAAN MOHAMMED KHALANDER

A handwritten signature in black ink, appearing to read 'Amaan MK', with a stylized, cursive script.

Date: 23 / September / 2026

Abstract

This report documents two practical exercises for the Robotics Software and Programming module. Task 1 builds a semantically labelled PyBullet simulation of a small inspection and light-warehousing room, using five custom URDF object categories, a two-wheel robot, fourteen placed objects, six camera viewpoints, and a dictionary-based semantic registry supporting role lookup and nearest-target queries. Task 2 implements a six-state finite state machine for a TurtleBot3 in ROS2, running in a custom Gazebo warehouse world with eight pallet obstacles, four patrol checkpoints and a charging dock, adding path planning, LIDAR and odometry based obstacle avoidance, a keyboard interaction node, and a monitoring node that logs state transitions and mission metrics. Task 2 was run to completion on TheConstruct.ai inside Rosject XctZpczWOA, with a local ROS2 Jazzy and Gazebo Harmonic workspace used only as a supplementary aid. Together, the two **tasks** trace one thread: representing what objects and behaviours mean, first as static metadata attached to a scene, then as a state machine reasoning about sensor data in real time.

Introduction

The Robotics Software and Programming module asks students to design, build and evaluate robotic software entirely within simulation, so the resulting skills transfer to hardware without requiring access to it. The brief, Dynamic Robotics Simulation and Autonomous System Design, sets out two stages of increasing complexity: an interactive PyBullet environment with object-level semantics, and a ROS2-based autonomous system with sensing, navigation, human interaction and self-monitoring. The stages use different toolchains, but the submission guidelines require one consolidated report, so both are presented together here using the official BSBI template.

Task 1 treats a simulated room as more than geometry: every object is assigned a role, obstacle, target, landmark, shelf or charger, stored so it can be queried in code rather than inferred visually. Task 2 carries the idea into a dynamic setting, where a mobile robot reads LIDAR and odometry in real time, chooses between discrete behavioural states, and reports what it did and why. The thread running through the report is the progression from meaning attached to objects to meaning attached to behaviour.

Task 1: Interactive PyBullet Environment with Object Semantics

3.1 Project Definition

The objective of Task 1 was a structured, semantically meaningful PyBullet scene in which the roles of objects, not just their geometry, could be queried programmatically. Before any code was written the room was sketched on paper as a six by four metre space: a single entrance, a central aisle, storage shelving along the back wall, a charging dock near the entrance, and reference landmarks at each corner. That sketch was then converted directly into URDF categories and object coordinates, so every placed object traces back to a deliberate position rather than an arbitrary grid.

The finished scene holds fourteen object instances across five URDF categories, above the assignment's minimum of eight objects and four categories, plus a two-wheel robot repositioned between camera renders to demonstrate movement through the space. All object geometry was authored directly as URDF primitives (boxes and cylinders) rather than imported meshes, keeping the notebook self-contained. Richer, photorealistic indoor datasets such as SceneNet RGB-D (McCormac et al., 2017) were considered but set aside as a direction for future extension.

3.2 Literature Review

Three strands informed Task 1: simulators, semantic representation and labelled scenes.

- **Simulators.** PyBullet exposes the Bullet physics engine through Python, loads URDF models and includes a CPU renderer (Coumans and Bai, 2021; PyBullet, 2023), so the notebook runs headless in Colab. Collins et al. (2021) review simulators across robotics; Task 1 needs scene construction and rendering rather than contact-rich manipulation, which favours PyBullet's light setup.
- **Semantic representation.** Kostavelis and Gasteratos (2015) survey semantic mapping for mobile robots, noting its role in human-robot interaction, knowledge representation and planning. The registry built here is the simplest form of that idea: designer-assigned metadata rather than an inferred map, letting code ask what an object is, not only where it is.
- **Labelled scenes.** McCormac et al. (2017) generated SceneNet RGB-D, a large synthetic indoor dataset with ground-truth labels. Task 1 is a far smaller, hand-authored version, trading photorealism for full control over labels. Duckett et al. (2018) note that field robots

must interpret unstructured surroundings reliably, so explicit roles matter beyond simulation.

Task 1's labels are therefore asserted rather than inferred, so it demonstrates query mechanics rather than perception (Section 3.6).

3.3 Design and Implementation

The notebook (Task1_PyBullet_Object_Semantics.ipynb) is a pipeline of nine executable cells. A setup cell imports pybullet, pybullet_data, numpy and matplotlib, installing pybullet automatically if missing, whether opened in Google Colab or run locally. A generation cell writes six URDF files to disk: an obstacle (crate), a target zone, a landmark, a shelf, a charging dock, and a two-wheel robot with a box chassis and two cylindrical wheels. An environment cell connects to PyBullet in DIRECT mode (no GUI, fully scriptable), loads a ground plane from the bundled pybullet_data assets, and instantiates the fourteen objects plus the robot at the coordinates taken from the floor sketch: four crate obstacles flanking the aisle, three target zones (pickup, an off-aisle inspection point, and drop-off), four corner landmarks, two back-wall shelves, and one charging dock. Figure 1 summarises this pipeline end to end.

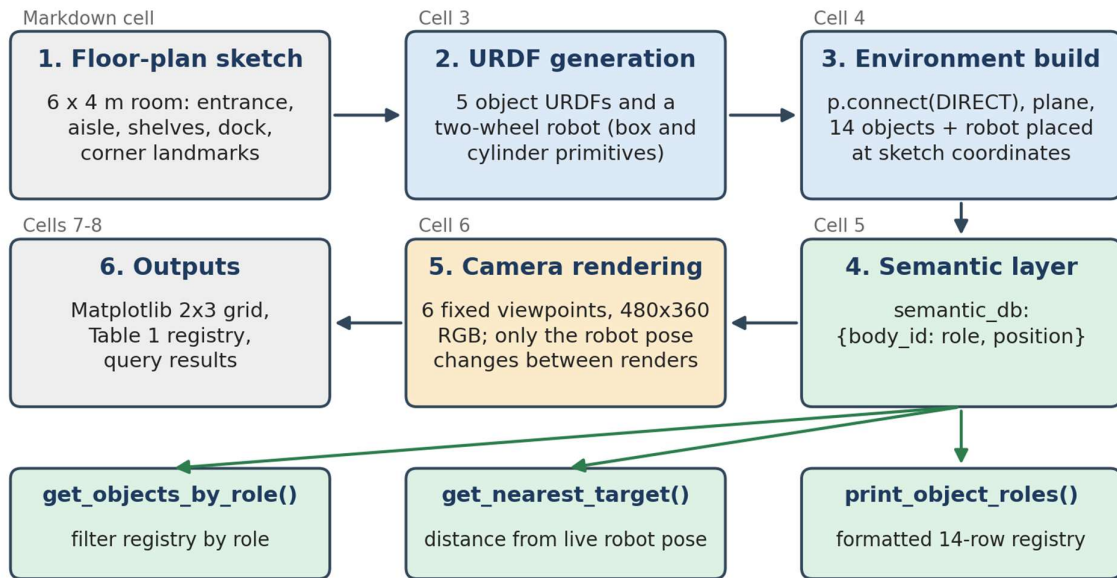


Figure 1: Task 1 pipeline, from the floor-plan sketch and URDF generation through the semantic layer and camera rendering to the query helpers and outputs.

A semantic layer, `semantic_db`, is a plain Python dictionary keyed by PyBullet body id, mapping each id to its role string and (x, y, z) position; Table 1 reproduces this registry exactly as printed. Three helper functions operate on it: `get_objects_by_role`, which filters by role string;

`get_nearest_target`, which computes Euclidean distance from the robot’s current base position to every target-labelled object and returns the closest; and `print_object_roles`, which prints a formatted summary of the whole registry. A rendering cell configures six camera viewpoints (the brief requires three; six were rendered as a deliberate extension) with `computeViewMatrix` and `computeProjectionMatrixFOV` (PyBullet, 2023), repositioning only the robot between renders while every other object stays fixed, then calls `getCameraImage` to capture a 480 by 360 RGB array per viewpoint.

The principal tools were Python 3, PyBullet for physics and rendering, NumPy for array handling, and Matplotlib for visualisation. No learned perception model was used: roles are assigned deterministically at construction time, which keeps the demonstration self-contained but means it does not infer role from image content alone, a limitation returned to in Section 3.6.

Table 1: Semantic object registry as printed by `print_object_roles()` (14 objects, excluding the robot).

body_id	role	position (x, y, z) m
1	obstacle	(1.50, 0.80, 0.25)
2	obstacle	(1.50, -0.80, 0.25)
3	obstacle	(3.50, 0.80, 0.25)
4	obstacle	(3.50, -0.80, 0.25)
5	target	(0.50, 0.00, 0.01)
6	target	(3.00, 1.60, 0.01)
7	target	(5.50, 0.00, 0.01)
8	landmark	(0.30, 1.80, 0.70)
9	landmark	(0.30, -1.80, 0.70)
10	landmark	(5.70, 1.80, 0.70)
11	landmark	(5.70, -1.80, 0.70)
12	shelf	(5.50, 1.00, 0.80)
13	shelf	(5.50, -1.00, 0.80)
14	charger	(0.30, 0.90, 0.07)

3.4 Code Implementation

The system is implemented as a Python Jupyter notebook, so results are identical whether run locally or in Colab. The excerpts below are abridged from the notebook.

URDF generation (cell 3): each category is a short URDF string written to disk; the obstacle is shown.

```
OBSTACLE_URDF = """<?xml version="1.0"?>
<robot name="obstacle">
  <link name="base_link">
    <visual><geometry>
      <box size="0.5 0.5 0.5"/></geometry></visual>
    <collision><geometry>
      <box size="0.5 0.5 0.5"/></geometry></collision>
    <inertial><mass value="0"/></inertial>
  </link>
</robot>"""
URDF_PATHS["obstacle"] = write_urdf("obstacle.urdf", OBSTACLE_URDF)
```

Semantic layer and query (cell 5): semantic_db maps body ids to roles; get_nearest_target reads the robot's live pose and returns the closest target.

```
semantic_db = {}
for body_id, role, pos in object_ids:
    semantic_db[body_id] = {"role": role, "position": pos}

def get_nearest_target(robot_body_id):
    robot_pos, _ = p.getBasePositionAndOrientation(robot_body_id)
    targets = get_objects_by_role("target")
    best, best_dist = None, float("inf")
    for body_id, pos in targets:
        dist = math.dist(robot_pos, pos)
        if dist < best_dist:
            best_dist, best = dist, (body_id, pos)
    return {"body_id": best[0], "position": best[1],
            "distance": best_dist}
```

Camera rendering (cell 6): fixed eye and target pairs are rendered on the CPU in DIRECT mode; only the robot's pose changes between calls.

```
def render_view(camera_eye, camera_target, up_vector=(0, 0, 1)):
    view = p.computeViewMatrix(camera_eye, camera_target, up_vector)
    proj = p.computeProjectionMatrixFOV(
        fov=60, aspect=IMG_WIDTH / IMG_HEIGHT,
        nearVal=NEAR_PLANE, farVal=FAR_PLANE)
    _, _, rgb, _, _ = p.getCameraImage(
        IMG_WIDTH, IMG_HEIGHT, view, proj,
        renderer=p.ER_TINY_RENDERER)
    return np.reshape(rgb, (IMG_HEIGHT, IMG_WIDTH, 4))[:, :, :3]
```

3.5 Execution and Testing

The notebook's outputs were generated in a single continuous kernel session (execution counts 28 to 36), first in a local Python environment. The completed notebook is hosted publicly on GitHub at github.com/Amaan9136/bsbi-assignments, under Sem2/Robotics Software and

Programming/task1-code/Task1_PyBullet_Object_Semantics.ipynb, and can be opened directly in Google Colab via https://colab.research.google.com/github/Amaan9136/bsbi-assignments/blob/main/Sem2/Robotics%20Software%20and%20Programming/task1-code/Task1_PyBullet_Object_Semantics.ipynb, which loads the same file straight from the public repository rather than a separately uploaded copy. Each of the nine cells ran without error and produced the outputs reproduced below; a fresh, cold-start rerun specifically inside Colab, confirming the setup cell's automatic pybullet install works end to end on that platform, has not yet been captured and is listed in the closing section.

Figure 2 shows all six rendered viewpoints with the robot pose used for each. Viewpoint 1 is an overhead-style view from the entrance with the robot at the origin; viewpoints 2 and 3 move progressively down the aisle towards the drop-off target; the remaining three take oblique angles from different corners of the room, so between them every object and every robot pose is visible from at least one direction.

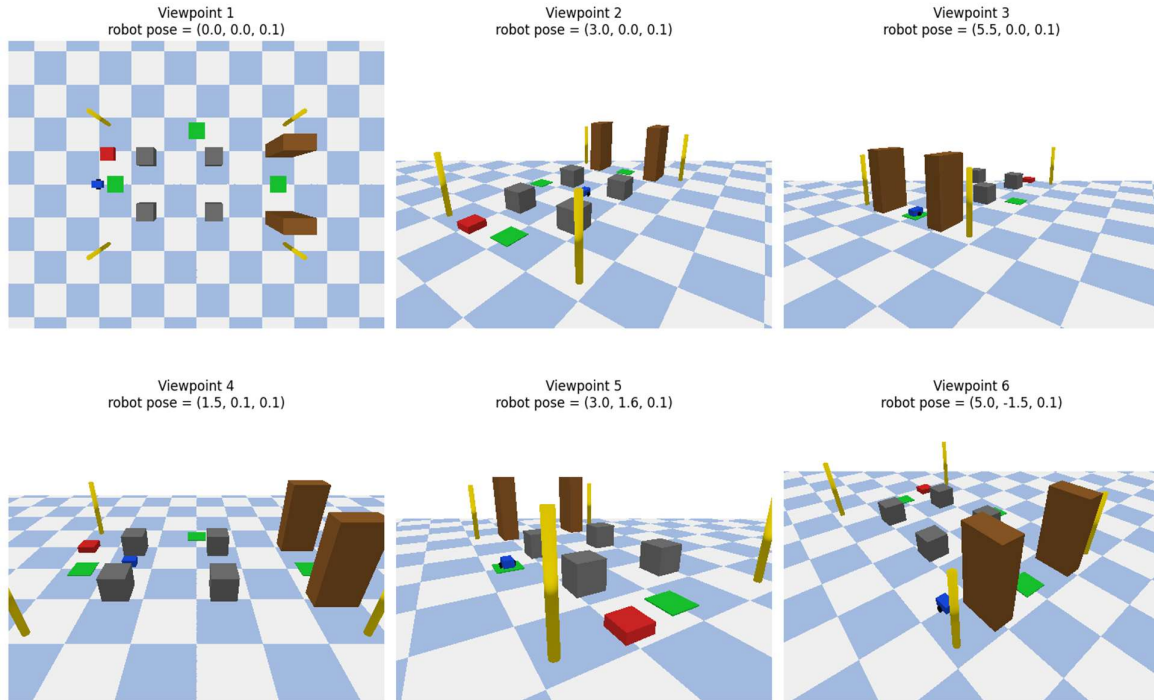


Figure 2: Six PyBullet camera viewpoints, robot repositioned between renders while all other objects remain fixed.

The interaction demonstration in cell 8 repositioned the robot to (5.5, 0.0, 0.1), close to the drop-off target, and exercised all three helper functions. `print_object_roles` reproduced the full fourteen-row registry in Table 1. `get_objects_by_role("obstacle")` correctly returned the four crate

obstacles (ids 1 to 4), and `get_objects_by_role("shelf")` returned the two shelves (ids 12 and 13). `get_nearest_target` correctly identified body id 7, the drop-off target at (5.50, 0.00, 0.01), as nearest, reporting a distance of 0.092 m, consistent with the robot’s near-target placement.

3.6 Evaluation and Reflection

The main challenge was keeping camera composition consistent while the robot moved. An early version computed each viewpoint’s eye and target position relative to the robot’s own pose, causing the framing to shift unpredictably and, in one configuration, letting the robot leave the frame entirely. This was resolved by fixing camera positions relative to the room’s known bounding extent and moving only the robot between renders; the viewpoint count was subsequently extended from the minimum of three to six so the semantic layer had a wider set of perspectives to be checked against. A second approach tried and rejected assigned roles by matching each object’s URDF filename with string patterns at load time; this was replaced with an explicit role argument recorded directly in the scene layout, less error-prone and auditable in one place.

A clear limitation is that semantic labels remain static metadata assigned by the designer, rather than inferred from the rendered images. This suits Task 1’s brief, which asks for an interpretable environment rather than a perception system, but stops short of recognising a role from appearance. A natural next step would train a lightweight classifier on SceneNet RGB-D (McCormac et al., 2017) to predict role from the images captured in Section 3.5. In real-world terms, the scenario maps onto warehouse and agricultural use cases, where a platform must distinguish an obstacle from a target and a landmark (Duckett et al., 2018); misclassification has operational consequences, so keeping the geometry-to-role mapping explicit and auditable matters.

Task 2: Goal-Oriented Autonomous Robot with Monitoring (ROS2)

4.1 Project Definition

The use case adopted for Task 2 is a warehouse inspection patrol robot: a TurtleBot3 that visits a closed loop of checkpoints around a warehouse bay, avoids pallet obstacles along its route, accepts simple operator commands, and reports on its own performance. This scenario was chosen because it exercises every capability the brief asks for, goal-oriented navigation, obstacle avoidance, human-robot interaction and self-monitoring, within one coherent mission rather than as disconnected demonstrations.

The package, `semantic_nav_monitor`, targets TheConstruct.ai running ROS2 Humble with Gazebo Classic and the standard TurtleBot3 stack, the platform the brief specifies. Development and the evidence in this section were carried out directly on TheConstruct.ai's browser-based ROSject platform, inside Rosject **XctZpczWOA** (URL identifier 1051220, app.theconstruct.ai/desktop/rosject/1051220), workspace `ros2_ws`. The free tier does not issue a permanent shareable URL, so the project was instead made public under its title, **XctZpczWOA**, which is the unique identifier required by the brief. Alongside this, an equivalent local Ubuntu 24.04 workspace running ROS2 Jazzy and Gazebo Harmonic (`ros_gz_sim`) was used optionally, purely to exercise GUI features that are easier to inspect outside the Rosject browser window, specifically the native LIDAR ray visualisation, the raw camera feed panel, and the planned-path line toggle; the package therefore ships a second, Harmonic-compatible launch file and world for that purpose, but it is a supplementary verification aid rather than the submission platform.

The custom Gazebo world, `warehouse_inspection` (built as both a Classic `.world` for TheConstruct.ai and a Harmonic `.sdf` for local verification), defines a walled rectangular bay containing eight pallet-stack obstacles (two per patrol leg) plus three smaller centreline pylons, four coloured checkpoint markers (one doubling as the charging dock), and no reliance on remote model assets. The robot model is a TurtleBot3 Burger fitted with a forward camera (`burger_cam`), equipped with a 360-degree LIDAR publishing `/scan` and wheel odometry publishing `/odom`.

4.2 Design and Implementation

The package contains four ROS2 nodes, summarised in Table 2. `mission_controller` is the only node that commands the robot; `monitor_node` is a purely passive observer; `keyboard_hri_node` provides single-keypress human interaction without requiring the Enter key; and `path_visualizer_node` draws the planned route inside the Gazebo client as a visual aid, with no effect on the robot's actual behaviour. Figure 3 shows the resulting node and topic architecture.

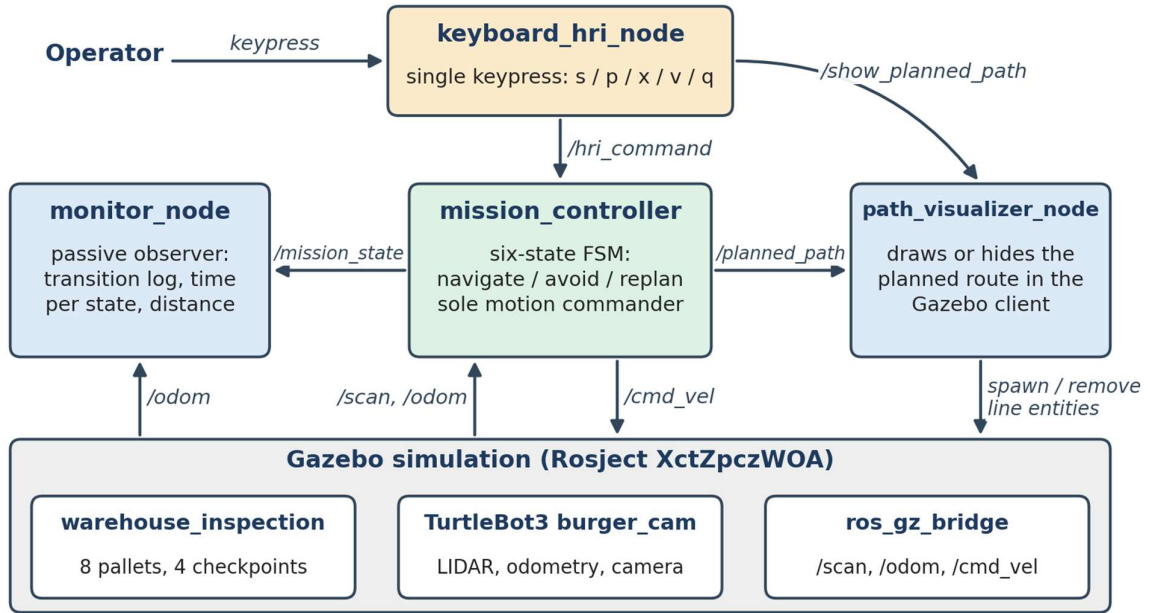


Figure 3: ROS2 node and topic architecture of the `semantic_nav_monitor` package, showing operator input, the four nodes and the Gazebo simulation.

Table 2: ROS2 node architecture.

Node	Subscribes	Publishes	Role
<code>mission_controller</code>	<code>/scan</code> , <code>/odom</code> , <code>/hri_command</code>	<code>/cmd_vel</code> , <code>/mission_state</code> , <code>/planned_path</code>	Six-state FSM: navigation, obstacle avoidance, replanning
<code>monitor_node</code>	<code>/mission_state</code> , <code>/odom</code>	(console log only)	Logs transitions; computes time-per-state, distance, obstacle count
<code>keyboard_hri_node</code>	terminal keypresses	<code>/hri_command</code> , <code>/show_planned_path</code>	Single-key HRI: s start, p pause, x stop, v toggle path, q quit

Node	Subscribes	Publishes	Role
path_visualizer_node	/planned_path, /show_planned_path	spawns/removes Gazebo line entities	Draws or hides the planned route in the Gazebo client

mission_controller implements six states rather than the assignment's minimum of five, adding a Planning state ahead of Navigate. Table 3 sets out each state's trigger and behaviour, matching the MissionState enum in the code exactly. Figure 4 shows the same machine as a transition diagram.

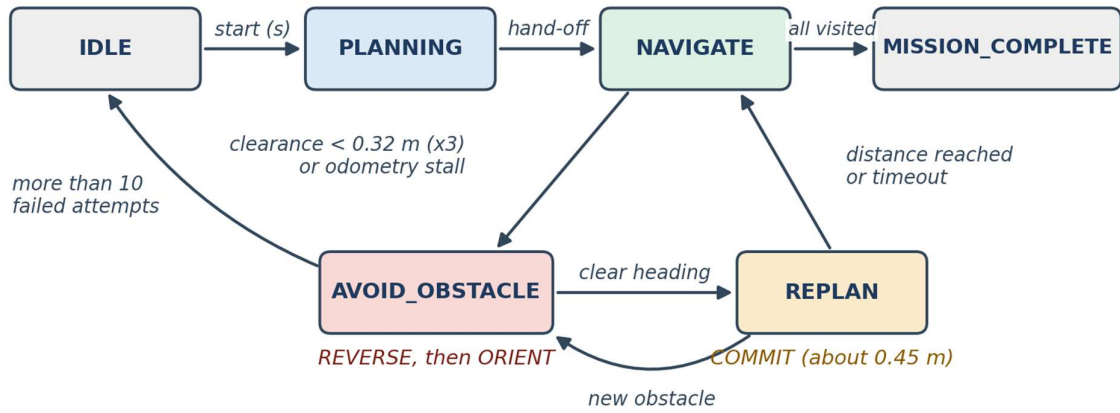


Figure 4: Six-state finite state machine in mission_controller, with the REVERSE, ORIENT and COMMIT sub-phases shown beneath their parent states.

Table 3: Finite state machine (six states).

State	Entered when	Behaviour
IDLE	At launch, or after a stop/abort	Stationary, waiting for a start command
PLANNING	start received	Builds the four-checkpoint route as a nav_msgs/Path, publishes it once, then hands off immediately
NAVIGATE	After planning, or once an obstacle is cleared	Proportional controller drives towards the next checkpoint
AVOID_OBSTACLE	Forward clearance below 0.32 m for three consecutive control cycles, or an odometry-based stall	Reverses briefly (REVERSE), then rotates to a LIDAR-probed clear heading (ORIENT)
REPLAN	A clear heading has been reached	Drives forward roughly 0.45 m along that heading (COMMIT) before resuming Navigate

State	Entered when	Behaviour
MISSION_COMPLETE	All four checkpoints visited, or stop received	Robot halts; final state published

Two design decisions reflect real engineering trade-offs. First, velocity commands are published as TwistStamped rather than plain Twist, since the `ros_gz_bridge` instance the TurtleBot3 launch infrastructure starts expects a stamped message; the unstamped type would leave the bridge silently unconnected. Second, obstacle avoidance is a three-phase sub-procedure (REVERSE, ORIENT, COMMIT): the robot probes escape headings at eleven angles per side via LIDAR, scores them by goal alignment and clearance, and commits only to one clearing a runway just above its own half-width (0.11 m) plus a small margin (0.05 m). An odometry-based stall check runs alongside: under 0.05 m of displacement after one second of commanded forward motion is treated as blocked regardless of LIDAR. After ten failed avoidance attempts, the mission aborts safely to IDLE.

`monitor_node` subscribes to `/mission_state` and `/odom`. On every state change it logs the transition with a timestamp, accumulates time in the previous state, and, specifically on entering `AVOID_OBSTACLE`, increments an obstacle-encounter counter, which the code's own documentation records as standing in for the assignment's "collisions" metric, since the robot avoids contact rather than colliding. Odometry feeds a running distance-travelled total, and a five-second timer prints a running summary throughout the mission. `path_visualizer_node` listens on `/planned_path` and, once told to show it, spawns thin static box entities along the route via the same Gazebo EntityFactory service used to spawn the robot; the line toggles from the keyboard ('v') or a native GUI panel, without altering the plan.

4.3 Execution and Testing

The package was built inside the Rosject with `colcon build --packages-select semantic_nav_monitor`, sourced, and launched after setting `TURTLEBOT3_MODEL=burger_cam`. Figure 5 shows the Rosject's three-pane workspace: the file explorer confirming the Rosject name **XctZpczWOA**, `mission_controller.py` in the code editor, the Simulation panel with the warehouse world, robot, eight pallets and four checkpoints, and a shell streaming `monitor_node`'s summary, confirming platform, code and a live run together.

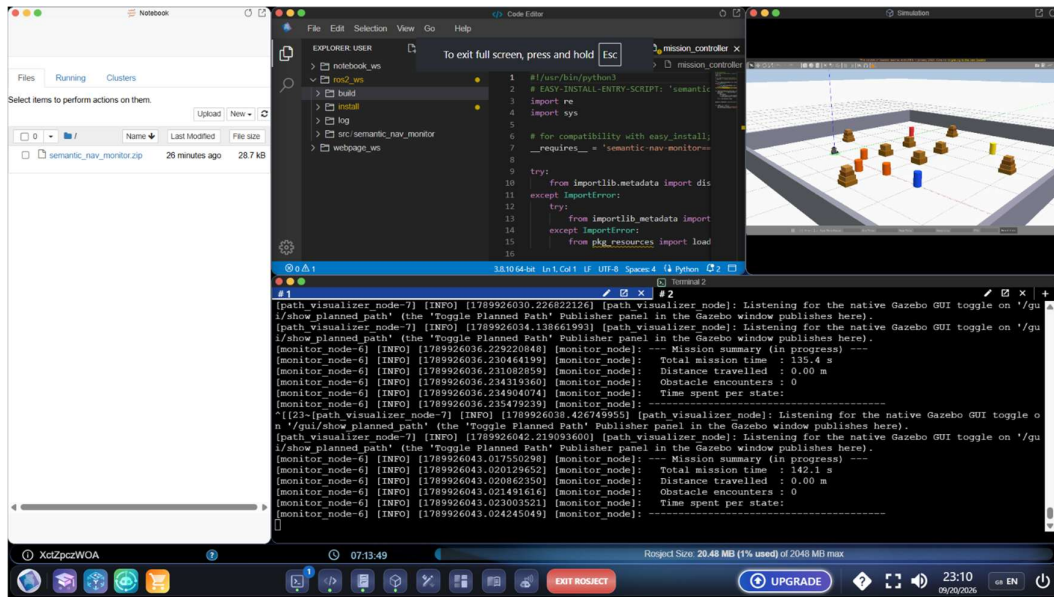


Figure 5: TheConstruct.ai Rosject XctZpczWOA: file explorer, mission_controller.py in the code editor, the live Simulation panel, and monitor_node's summary stream in the shell.

Figure 6 shows the Gazebo client mid-patrol, with the robot navigating among the pallets; the status bar's Real Time and Iterations counters (00:02:11 elapsed, 94,858 iterations) confirm a continuously running simulation, not a static load.

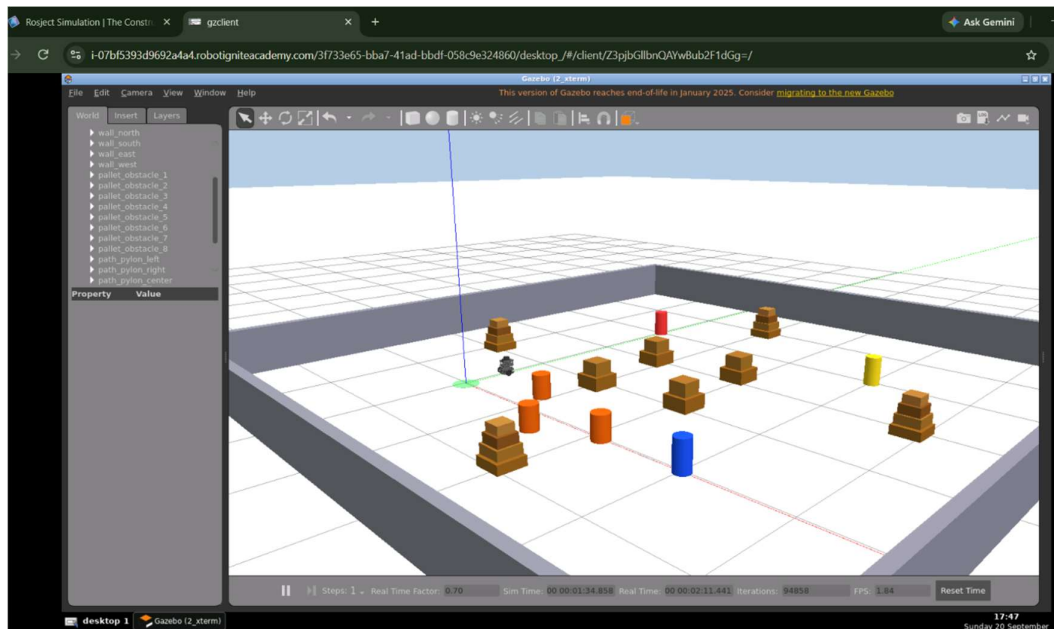


Figure 6: Gazebo client mid-patrol on TheConstruct.ai: robot among the eight pallet obstacles and checkpoint markers, with elapsed real time and iteration count visible.

Figure 7 shows `keyboard_hri_node` in a Rosject shell beside the live Simulation panel: `s` and `p` keypresses are echoed as Published HRI command: 'start' and 'pause', with Planned-path line: ON confirming the path toggle also responded, evidencing the human-robot interaction requirement directly.

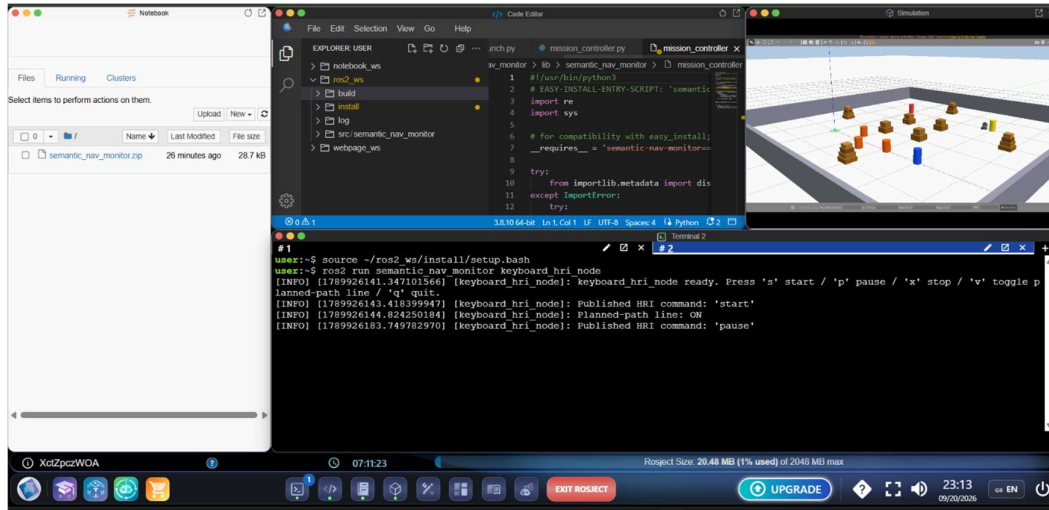


Figure 7: keyboard_hri_node in a Rosject shell: start and pause keypresses published and confirmed, alongside the live Simulation panel.

Figure 8 is a live `ros2` topic echo `/mission_state` capture. Reading oldest to newest, it records AVOID_OBSTACLE, REPLAN, AVOID_OBSTACLE, IDLE, PLANNING, NAVIGATE and finally MISSION_COMPLETE: the robot was mid-avoidance, returned briefly to IDLE, most likely paused and restarted via the keyboard node in Figure 7, then re-planned, navigated and completed, the clearest direct evidence of a full mission carried through on the required platform.

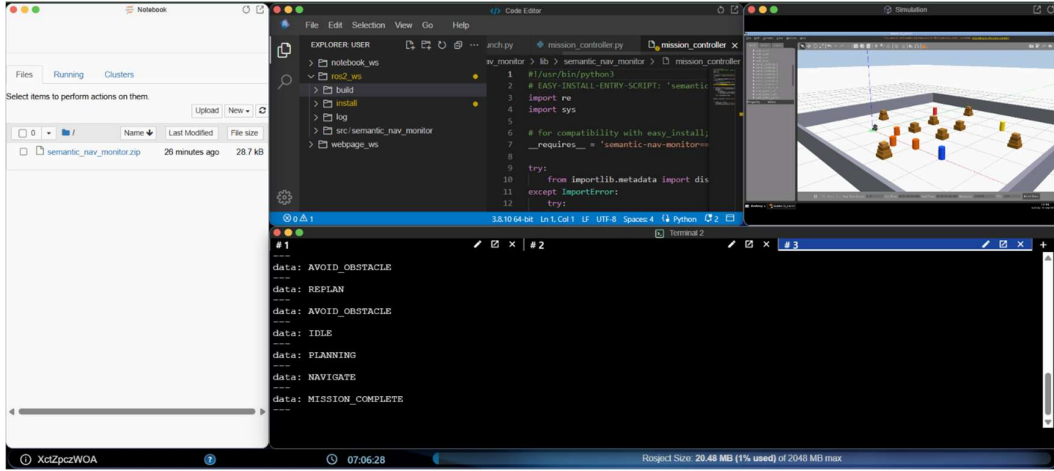


Figure 8: Live /mission_state echo showing the robot cycle through AVOID_OBSTACLE, REPLAN, IDLE, PLANNING, NAVIGATE and MISSION_COMPLETE.

Figure 9 captures monitor_node’s periodic summary shortly before that completion, reproduced in Table 4. As the header still reads “in progress” at capture, these are the last recorded progress figures rather than a separate post-completion summary, logged shortly before the MISSION_COMPLETE transition in Figure 8.

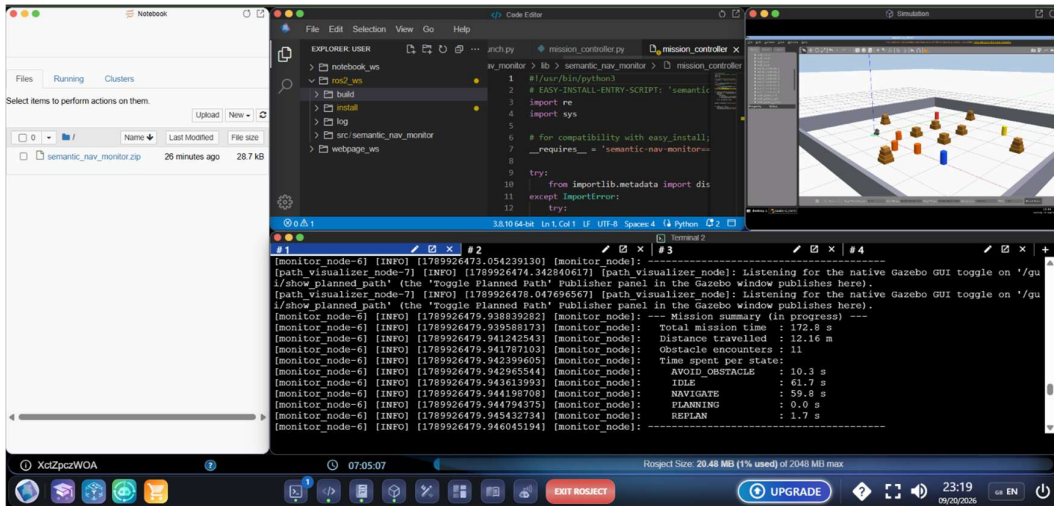


Figure 9: monitor_node’s mission summary, captured shortly before MISSION_COMPLETE.

Table 4: Mission summary logged by monitor_node on TheConstruct.ai, Rosject XctZpczWOA, shortly before MISSION_COMPLETE.

Metric	Value
Total mission time	172.8 s
Distance travelled	12.16 m
Obstacle encounters	11
Time in AVOID_OBSTACLE	10.3 s
Time in IDLE	61.7 s
Time in NAVIGATE	59.8 s
Time in PLANNING	0.0 s
Time in REPLAN	1.7 s

The local Jazzy/Harmonic workspace supplied one further, supplementary view not easily captured inside the Rosject browser window: Figure 10 shows native LIDAR-ray visualisation active near a pallet obstacle, with the forward camera feed in the same panel, confirming the sensing picture the AVOID_OBSTACLE logic in Section 4.2 acts on.

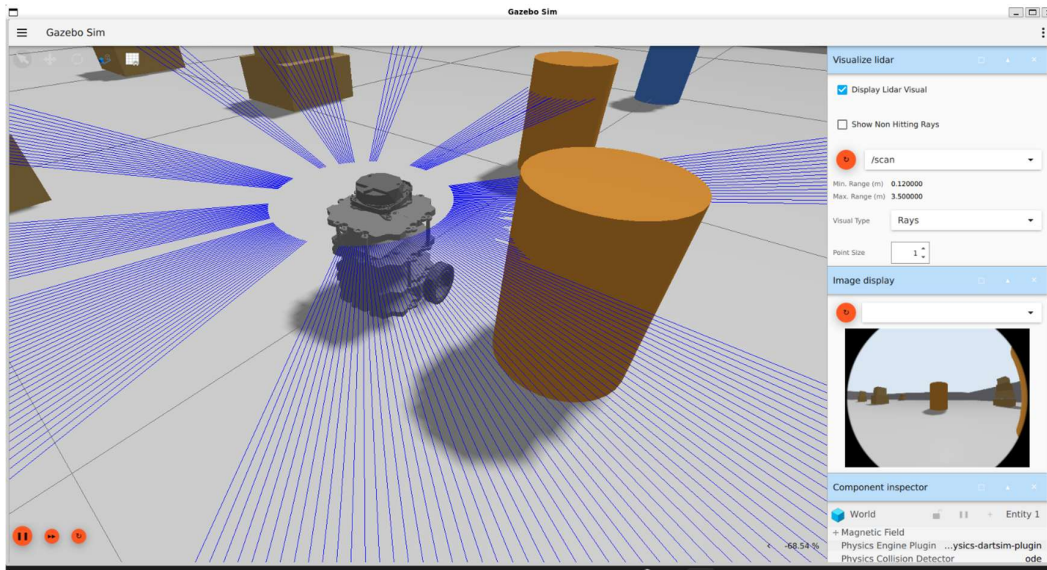


Figure 10: Local Gazebo Harmonic verification: LIDAR rays visualised around the robot near a pallet obstacle, with the forward camera feed shown alongside.

Five test cases are directly evidenced on the required platform: Rosject identity and a live simulation (Figure 5); a continuously running world (Figure 6); keyboard HRI with confirmed acknowledgement (Figure 7); a complete state cycle reaching MISSION_COMPLETE (Figure 8); and logged mission metrics (Figure 9, Table 4). LIDAR sensing is additionally evidenced locally (Figure 10).

4.4 Evaluation and Reflection

Table 4 is informative about a limitation met during development: eleven obstacle encounters against 12.16 m travelled, with 61.7 s in IDLE, points to at least one operator pause and restart, consistent with Figure 8. Several rounds of tuning were needed: the required escape-heading clearance was originally set well above the robot’s half-width, causing detours around gaps it could fit through, and an earlier scoring rule always picked the heading most aligned with the goal even at minimum clearance, causing oscillation at gap edges. Both were corrected by trimming margins closer to the real 0.11 m half-width and preferring, among similarly aligned headings, the one with more open space; only 10.3 s was ultimately spent in AVOID_OBSTACLE against 59.8 s in NAVIGATE.

A second limitation is the REPLAN state’s simplicity: it drives forward along a probed-clear heading for a fixed distance rather than computing a genuine path around the obstacle’s full extent. A local costmap-based planner, as used within the Nav2 navigation stack (Macenski et al., 2020), would route around extended obstacles more efficiently than this probe-and-commit approach, at the cost of considerably more implementation complexity. A third, procedural limitation is that TheConstruct.ai’s free tier issues no permanent shareable simulation URL, so the Rosject is shared by its unique name, XctZpczWOA, a platform constraint rather than a shortcoming of the package.

Despite these limitations, the underlying approach, a small number of explicit states, reactive obstacle handling backed by a redundant odometry check, and continuous self-reporting, maps directly onto real warehouse and service robotics, where operational staff generally need to understand why a robot took a particular action rather than trust an opaque policy (Siegwart, Nourbakhsh and Scaramuzza, 2011). The obstacle-encounter count and per-state timing in Table 4, logged on the platform the brief specifies and culminating in a genuine MISSION_COMPLETE transition, are exactly the kind of evidence a site operator would want before trusting a patrol robot in a live warehouse.

Combined Evaluation and Reflection

Together, the two tasks address one question: how a robotic system represents and acts on the meaning of what is around it. Task 1 demonstrates semantic representation in a static scene, where roles are assigned by the designer and retrieved by dictionary lookup, as the registry in Table 1 and the sub-tenth-of-a-metre nearest-target result in Section 3.5 show. Task 2 demonstrates semantically informed behaviour in a dynamic setting, where the robot selects among six states whose meaning, avoiding, replanning, completing, is a label it applies to its own behaviour.

Working through both tasks reinforced that simulation, perception, navigation and monitoring are stages of one pipeline. The camera observation in Task 1 and the LIDAR sensing in Task 2 both feed a decision layer, whether query functions or a state machine, and both benefit from an explicit monitoring mechanism that lets internal reasoning be inspected. The most instructive moment was a documented failure: the over-cautious obstacle avoidance in Task 2, corrected across several rounds of threshold tuning, showed that a system can avoid every obstacle yet still be a poor design, and catching this required the monitoring layer itself.

Concluding Remarks

This report has documented a PyBullet simulation with object-level semantics stored as static metadata and queried through helper functions, and a ROS2 goal-oriented navigation system for a TurtleBot3 with a six-state machine, LIDAR and odometry based obstacle avoidance, keyboard interaction and continuous self-monitoring. Both were implemented and executed against the brief, and the evidence presented is drawn from what each system actually produced: a six-viewpoint render set and object registry for a publicly hosted Task 1 notebook, and Rosject, world, interaction, state-cycle and mission-metric screenshots captured on TheConstruct.ai for Task 2, which ran to a genuine `MISSION_COMPLETE` inside Rosject XctZpczWOA. The one outstanding item is procedural: a cold-start confirmation of the Task 1 notebook inside Colab, listed below. The shared lesson is that semantic structure, whether attached to objects or expressed through a robot's own states, only becomes trustworthy once it is monitored and logged.

Acknowledgement

Perplexity AI (2026) was used to support literature searching, identify potentially relevant academic sources, and assist with grammar correction and word-limit reduction during editing.

All sources included in the reference list were checked by the author and cited as the original academic works rather than citing Perplexity AI as a source. The analysis, interpretation, discussion, and final written content submitted in this report are the author's own work and responsibility.

The architecture and pipeline diagrams in Sections 3.3 and 4.2 were generated using Python scripts with the matplotlib library, written specifically to render the structures described in this report. The scripts and their outputs were reviewed by the author before inclusion.

References

- Collins, J., Chand, S., Vanderkop, A. and Howard, D. (2021) 'A Review of Physics Simulators for Robotic Applications.' IEEE Access 9: 51416-51431. [online] Available from <https://doi.org/10.1109/ACCESS.2021.3068769> [Accessed 29 September 2026]
- Coumans, E. and Bai, Y. (2021) PyBullet, a Python Module for Physics Simulation for Games, Robotics and Machine Learning. [online] Available from <https://github.com/bulletphysics/bullet3> [Accessed 20 September 2026]
- Duckett, T., Pearson, S., Blackmore, S. and Grieve, B. (2018) Agricultural Robotics: The Future of Robotic Agriculture. UK-RAS White Papers. [online] Available from <https://arxiv.org/abs/1806.06762> [Accessed 20 September 2026]
- Kostavelis, I. and Gasteratos, A. (2015) 'Semantic mapping for mobile robotics tasks: A survey.' Robotics and Autonomous Systems 66: 86-103. [online] Available from <https://doi.org/10.1016/j.robot.2014.12.006> [Accessed 29 September 2026]
- Macenski, S., Martin, F., White, R. and Gines Clavero, J. (2020) 'The Marathon 2: A Navigation System.' In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). Las Vegas, 24-30 October 2020: 2718-2725
- McCormac, J., Handa, A., Leutenegger, S. and Davison, A.J. (2017) 'SceneNet RGB-D: Can 5M Synthetic Images Beat Generic ImageNet Pre-training on Indoor Segmentation?' In: Proceedings of the IEEE International Conference on Computer Vision (ICCV). Venice, 22-29 October 2017: 2678-2687
- Open-Source Robotics Foundation (2024) ROS 2 Documentation: Humble. [online] Available from <https://docs.ros.org/en/humble/index.html> [Accessed 20 September 2026]
- Open Robotics (2024) Gazebo Harmonic Documentation. [online] Available from <https://gazebo.org/docs/harmonic> [Accessed 20 September 2026]
- PyBullet (2023) PyBullet Quickstart Guide. [online] Available from <https://pybullet.org/wordpress/> [Accessed 20 September 2026]
- Quigley, M., Gerkey, B. and Smart, W.D. (2015) Programming Robots with ROS: A Practical Introduction to the Robot Operating System. Sebastopol, CA: O'Reilly Media

ROBOTIS (2023) TurtleBot3 e-Manual. [online] Available from <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/> [Accessed 20 September 2026]

Siegwart, R., Nourbakhsh, I.R. and Scaramuzza, D. (2011) Introduction to Autonomous Mobile Robots. 2nd ed. Cambridge, MA: MIT Press

TheConstruct (2023) ROS2 Basics in 5 Days (Python). [online] Available from <https://www.theconstruct.ai/> [Accessed 20 September 2026]